Ascent
FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

Cours apprenant

ReST API — bonnes pratiques et sécurité

MODULE 6
Performances et scalabilité
Mesure, cache, compression, load balancing, throttling, résilience

CRÉATEUR DU COURS
Boris Rose
Formateur & concepteur du contenu

Table des matières

Avant-propos **3**

1 Mesurer la performance **3**
1.1 Indicateurs essentiels . . . 3
1.2 Pourquoi parler en percentiles, pas en moyenne . . . 3
1.3 Lois fondamentales . . . 3
1.3.1 Loi de Little . . . 3
1.3.2 Loi d'Amdahl . . . 4
1.3.3 Universal Scalability Law (USL) . . . 5
1.4 Vocabulaire système à comprendre avant d'aller plus loin . . . 5

2 Cache à plusieurs étages **7**
2.1 Les 5 niveaux de cache . . . 7
2.1.1 Niveau 1 — Cache navigateur . . . 7
2.1.2 Niveau 2 — CDN / edge . . . 8
2.1.3 Niveau 3 — Reverse proxy . . . 8
2.1.4 Niveau 4 — Cache applicatif . . . 9
2.1.5 Niveau 5 — Cache base de données . . . 11
2.2 Stratégies de cache : quatre patterns . . . 13
2.2.1 Cache-aside (lazy loading) . . . 13
2.2.2 Read-through . . . 13
2.2.3 Write-through . . . 14
2.2.4 Write-behind (write-back) . . . 14
2.2.5 Variante — Refresh-ahead . . . 14
2.3 Cache HTTP : Cache-Control, ETag, Last-Modified . . . 15
2.3.1 En-tête `Cache-Control` . . . 15
2.3.2 En-tête `ETag` (Entity Tag) . . . 15
2.3.3 En-tête `Last-Modified` . . . 16
2.4 Cache stampede — piège à connaître . . . 17

3 Compression et connexions **17**
3.1 Compression HTTP . . . 17
3.1.1 gzip . . . 17
3.1.2 Brotli (`br`) . . . 17
3.1.3 Règles pratiques . . . 18
3.2 Versions de HTTP et performance . . . 18

4 Scalabilité horizontale **18**
4.1 Stateless = scalable . . . 18
4.2 Load balancing . . . 19
4.2.1 L4 vs L7 — deux couches de routage . . . 19
4.2.2 Algorithmes de répartition . . . 20
4.2.3 Health checks — le LB doit savoir qui est vivant . . . 21
4.2.4 Exemple complet : 3 backends + load balancer + bench . . . 22
4.3 Health checks . . . 23
4.4 Autoscaling . . . 23

5 API Gateways **23**

6 Throttling et rate limiting **24**
6.1 Cinq notions à distinguer . . . 24
6.2 Sur quel axe ? . . . 24
6.3 Algorithmes de rate limiting . . . 24
6.4 Headers HTTP standardisés . . . 24
6.5 Côté client : être un bon citoyen . . . 25
6.5.1 1. Respecter `Retry-After` . . . 25
6.5.2 2. Exponential backoff avec jitter . . . 25
6.5.3 3. Fonction de retry complète . . . 26
6.5.4 4. Lire `X-RateLimit-Remaining` pour anticiper . . . 27
6.5.5 5. Préférer les API en lot . . . 27

7 Patterns de résilience **27**
7.1 Timeout . . . 27
7.2 Retry + backoff + jitter . . . 27
7.3 Circuit breaker . . . 28
7.4 Bulkhead . . . 28
7.5 Fallback . . . 28
7.6 Idempotency-Key . . . 28

8 Les outils du module **28**
8.1 Redis . . . 28
8.2 Memcached . . . 28
8.3 nginx et Varnish . . . 28
8.4 Kong, Tyk, KrakenD, Apache APISIX . . . 29
8.5 k6 . . . 29
8.6 Outils CLI de bench HTTP . . . 29
8.6.1 wrk — débit maximal en ligne de commande . . . 29
8.6.2 hey — l'équivalent moderne d'Apache Bench . . . 30
8.6.3 Vegeta — scénarios constants et rapports HTML . . . 31
8.6.4 Comparatif rapide . . . 32
8.7 Cloudflare, Fastly, CloudFront, Akamai . . . 32
8.8 Resilience4j, Polly, opossum, tenacity . . . 32

9 Synthèse **32**

10 Lexique du module **33**

11 Pour aller plus loin **34**

Avant-propos

Ce cours est le compagnon écrit du **Module 6**, dernier de la formation. Il couvre la mesure des performances, les stratégies de cache, la scalabilité horizontale, le throttling et les patterns de résilience. *Cours conçu et rédigé par Boris Rose, formateur.*

1 Mesurer la performance

1.1 Indicateurs essentiels

Définition

Latence : temps qu'une requête met à être traitée, de l'envoi par le client à la réception de la réponse.

Débit (RPS) : nombre de requêtes par seconde que le système traite.

Concurrence : nombre de requêtes traitées en parallèle à un instant donné.

1.2 Pourquoi parler en percentiles, pas en moyenne

Définition

Percentile (p_N) : valeur en-deçà de laquelle se trouve N % des observations. Exemple : si $p_{95} = 200$ ms, cela signifie que 95 % des requêtes répondent en moins de 200 ms.

À retenir

La moyenne ment toujours. Si 99 % des requêtes répondent en 50 ms et 1 % en 5 secondes, la moyenne est d'environ 100 ms. La moyenne masque l'expérience désastreuse de 1 utilisateur sur 100.

Toujours raisonner en $p_{50}, p_{95}, p_{99}, p_{99.9}$.

1.3 Lois fondamentales

Trois lois mathématiques permettent de raisonner sur la capacité d'un système, sa scalabilité et ses limites. Les connaître évite de prendre des décisions architecturales naïves (« il suffit d'ajouter des serveurs ») qui se révéleront mauvaises sous charge réelle.

1.3.1 Loi de Little

Définition

Loi de Little (*John Little*, MIT, 1961) : dans un système en régime stationnaire, $L = \lambda \cdot W$, où L est le nombre moyen d'éléments dans le système, λ le taux d'arrivée (RPS) et W le temps moyen passé dans le système (latence). Loi universelle qui s'applique à toute file d'attente : caisse de supermarché, ligne de production, threads d'un serveur HTTP.

Application directe pour dimensionner une API :

$$\text{concurrence requise} = \text{débit} \times \text{latence moyenne}$$

Exemple

Cas n°1 — dimensionner un pool de workers.
Votre API doit absorber $\lambda = 500$ RPS avec une latence moyenne $W = 200$ ms $= 0{,}2$ s. La concurrence requise est :

$$L = 500 \times 0{,}2 = 100 \text{ requêtes en cours simultanément.}$$

Conclusion : il faut au moins 100 *workers* (ou threads, ou connexions) capables de traiter en parallèle. Avec 50 workers, vous aurez de la file d'attente et la latence va exploser.

Cas n°2 — estimer le débit max d'un système synchrone.
Un serveur Node mono-thread ne peut traiter *qu'une* requête à la fois en bloquant. Si chaque requête dure 50 ms :

$$\lambda_{\max} = \frac{L}{W} = \frac{1}{0{,}05} = 20 \text{ req/sec.}$$

Au-delà, la file s'accumule. C'est pourquoi on utilise des serveurs asynchrones (Node event loop, Go goroutines) qui n'ont pas cette limitation par requête.

Cas n°3 — prédire la latence à charge constante.
Vous mesurez $L = 80$ requêtes en cours et $\lambda = 200$ RPS. Alors :

$$W = \frac{L}{\lambda} = \frac{80}{200} = 0{,}4 \text{ s} = 400 \text{ ms.}$$

Si la latence cible est 100 ms, soit vous diminuez L (plus de workers, mais attention au CPU), soit vous diminuez λ (rate limiting), soit vous accélérez le traitement (cache, optimisations).

1.3.2 Loi d'Amdahl

Définition

Loi d'Amdahl (*Gene Amdahl*, IBM, 1967) : si une fraction p du temps de calcul est parallélisable et $1-p$ reste séquentielle, alors l'accélération maximale avec n unités de calcul est

$$\text{Speedup}(n) = \frac{1}{(1-p) + \frac{p}{n}}.$$

Quand $n \to \infty$, l'accélération tend vers $\frac{1}{1-p}$: la partie séquentielle plafonne le gain, peu importe combien on ajoute de cœurs ou de serveurs.

Exemple

Application concrète : votre pipeline de traitement comporte 95 % de calcul parallélisable et 5 % d'agrégation séquentielle finale.

- Avec 1 cœur : durée de référence ($1\times$).
- Avec 10 cœurs : $\frac{1}{0{,}05+0{,}095} = 6{,}9\times$ (au lieu de $10\times$ idéal).
- Avec 100 cœurs : $\frac{1}{0{,}05+0{,}00095} = 19{,}6\times$ seulement.
- Avec ∞ cœurs : $\frac{1}{0{,}05} = 20\times$ maximum, jamais plus.

Conséquence concrète : payer 100 cœurs pour gagner $19{,}6\times$ là où 20 cœurs en donnaient déjà $9{,}3\times$ n'est rentable que si le gain marginal vaut le coût. Au-delà, l'effort doit porter sur la réduction de la partie séquentielle (souvent : suppression de verrous, sharding de la BDD, file d'attente).

1.3.3 Universal Scalability Law (USL)

Définition

Universal Scalability Law (*Neil Gunther*, 1991) : raffinement d'Amdahl qui ajoute un terme de *contention* (parties séquentielles, α) *et* de *cohérence* (synchronisations entre nœuds, β). Formule :

$$\text{Speedup}(n) = \frac{n}{1 + \alpha(n-1) + \beta n(n-1)}.$$

Avec $\beta > 0$, la courbe de speedup **redescend** après un pic : ajouter des serveurs *dégrade* la performance globale, parce que la synchronisation devient plus coûteuse que le travail utile.

Exemple

Cas typique : un cluster Postgres avec réplication synchrone. Au-delà de 5–7 réplicas, chaque écriture doit être confirmée par toutes les répliques avant retour. Le gain marginal devient négatif. Beaucoup d'organisations « savent » empiriquement qu'au-delà de N serveurs leur base ralentit ; la USL formalise pourquoi.

Conséquence pratique : pour scaler très haut, il faut *partitionner* (sharding) pour casser la cohérence globale en cohérence locale. C'est ce que font DynamoDB, Cassandra, MongoDB Sharded : chaque shard est cohérent en interne, indépendant des autres.

Synthèse des trois lois : Little dimensionne, Amdahl plafonne le gain, USL explique pourquoi « plus de serveurs » peut dégrader la performance.

1.4 Vocabulaire système à comprendre avant d'aller plus loin

Les sections suivantes utilisent un vocabulaire que nous figons ici une bonne fois pour toutes.

Définition

Agrégation séquentielle finale : étape de fin de pipeline qui doit collecter et fusionner les résultats partiels de N travailleurs parallèles. Par essence, elle ne se parallélise pas (on attend tout le monde). C'est la « partie séquentielle » de la loi d'Amdahl.

Exemple : pour calculer le chiffre d'affaires national, 50 workers calculent en parallèle le CA par région ; un dernier processus additionne les 50 résultats. Cette dernière addition est l'agrégation séquentielle.

Définition

Verrou (*lock*) : mécanisme qui interdit à plusieurs processus ou threads d'accéder simultanément à une ressource partagée. Force la sérialisation : pendant qu'un acteur tient le verrou, les autres attendent.

Exemple : `SELECT ... FOR UPDATE` en SQL pose un verrou sur la ligne, le temps de la transaction. `Mutex`, `RWLock`, `Semaphore` sont les variantes en programmation. Trop de verrous = contention, faible scalabilité.

Définition

File d'attente (*queue*) : structure FIFO (*First In, First Out*) où les messages s'accumulent en attendant d'être traités par un consommateur. Sert à découpler un producteur rapide d'un consommateur plus lent et à lisser des pics de charge.

Exemple : RabbitMQ, Kafka, AWS SQS, Redis Lists. Une API qui reçoit un upload d'image renvoie immédiatement « OK » au client, dépose un message dans une file, et un worker en arrière-plan redimensionne l'image.

Définition

Sharding (de la BDD) : technique qui consiste à partitionner les données d'une grande base en plusieurs morceaux (*shards*), chacun hébergé sur un serveur distinct. La clé de partitionnement (*shard key*, par exemple `user_id`) détermine sur quel shard se trouve chaque ligne.

Exemple : 100 millions d'utilisateurs répartis sur 8 shards `user-shard-0` à `user-shard-7`, déterminés par `user_id mod 8`. L'utilisateur 42 vit sur le shard 2 ; l'utilisateur 99 sur le shard 3. Chaque shard est plus petit, plus rapide, et la charge se répartit.

Définition

Cluster Postgres : groupe de plusieurs instances Postgres qui coopèrent pour offrir une base unique. Une instance est *primaire* (accepte les écritures) ; les autres sont des *répliques* (acceptent les lectures, suivent le primaire). Outils : Patroni, Stolon, Postgres Operator (pour Kubernetes).

Définition

Réplica (*replica*) : copie d'une base de données qui se met à jour à partir d'un primaire (*master*). Les répliques servent à : encaisser plus de lectures (*read scaling*), survivre à une panne du primaire (*failover*), placer la donnée près des utilisateurs (réplique géographique).

Modes : *réplication synchrone* (le primaire attend que la réplique confirme l'écriture — plus sûr mais plus lent), *asynchrone* (le primaire répond immédiatement, la réplique rattrape — plus rapide mais perte possible en cas de crash).

Définition

Partitionner pour casser la cohérence globale en cohérence locale : phrase clé de la section USL. Signifie qu'au lieu de demander à un seul cluster d'être cohérent sur l'ensemble des données (coûteux, freine la scalabilité), on découpe les données en morceaux indépendants ; chaque morceau est cohérent en interne, et il n'existe *plus* de cohérence globale à maintenir.

Exemple : Twitter/X ne tient pas une seule grande table « tweets » cohérente avec le monde entier. Les tweets sont *shardés* par `user_id` ; un coin du système peut avoir des tweets que d'autres coins n'ont pas encore vus. La perception « globale » (le *timeline*) est reconstituée *a posteriori* par agrégation. Sacrifice de la cohérence forte pour la scalabilité — c'est le théorème CAP en pratique.

2 Cache à plusieurs étages

Définition

Cache : stockage temporaire d'une donnée pour la réutiliser sans la recalculer ou la re-télécharger. Permet de réduire la latence et la charge sur les systèmes en aval.

2.1 Les 5 niveaux de cache

Un cache n'est jamais unique : on les empile, du plus proche de l'utilisateur (navigateur) au plus profond (BDD). À chaque niveau, on évite la requête vers le niveau suivant et on gagne en latence *et* en charge.

2.1.1 Niveau 1 — Cache navigateur

Définition

Cache navigateur : stockage local que chaque navigateur (Chrome, Firefox, Safari, Edge) maintient automatiquement pour les ressources HTTP. Le serveur *ordonne* le cache via les en-têtes `Cache-Control`, `ETag`, `Last-Modified` ; le navigateur applique. Aucune ligne de code spécifique côté front : tout passe par les en-têtes HTTP.

Implémentation côté serveur (Express) :

Exemple

```
app.get("/articles/:id", async (req, res) => {
  const article = await db.articles.findById(req.params.id);
  if (!article) return res.status(404).end();

  // 1. Calculer un ETag stable (hash du contenu)
  const etag = '"' + crypto.createHash("sha1")
                       .update(JSON.stringify(article))
                       .digest("hex").slice(0, 16) + '"';

  // 2. Si le client a déjà cette version, retourner 304 sans corps
  if (req.headers["if-none-match"] === etag) {
    return res.status(304).end();
  }

  // 3. Sinon, renvoyer avec instructions de cache
  res.set("ETag", etag);
  res.set("Cache-Control", "public, max-age=300, must-revalidate");
  res.json(article);
});
```

Effet visible : un `F5` sur la page recharge l'article ; mais une navigation depuis un autre lien dans les 5 minutes le ressort du cache navigateur en quelques millisecondes, sans appel réseau.

2.1.2 Niveau 2 — CDN / edge

Définition

CDN (*Content Delivery Network*) : réseau de serveurs distribués géographiquement (PoP, *Points of Presence*) qui mettent en cache vos réponses au plus près des utilisateurs. Quand un Parisien interroge votre API, c'est un serveur Cloudflare à Roissy qui répond, pas votre origin à Dublin. Gain typique en latence : **100–200 ms** en moins.

Acteurs principaux :

- **Cloudflare** : leader mondial (gratuit pour usage modeste). Plus de 300 PoP.
- **Fastly** : très utilisé par les médias (NYT, BBC), excellent en cache d'API.
- **Amazon CloudFront** : intégré à AWS, économique si on est déjà chez eux.
- **Akamai** : pionnier historique, entreprise et streaming vidéo.
- **Bunny.net**, **KeyCDN** : alternatives *low-cost*.

Implémentation type (Cloudflare) :

Exemple

1. Pointer votre DNS `api.example.com` vers Cloudflare (*nameservers*).
2. Activer le *Proxy* (icône nuage orange).
3. Dans *Caching* → *Configuration* : définir une `Cache Rule` :

```
URI Path matches: /api/v1/products/*
Cache Eligibility: Eligible for cache
Edge Cache TTL: 5 minutes
Origin Cache Control: respect # honore vos en-têtes
```

Effet : votre API à Paris voit 90 % du trafic absorbé par Cloudflare (hit ratio typique). Seuls les 10 % restants atteignent votre origin.

Purge ciblée après une mise à jour :

```
curl -X POST "https://api.cloudflare.com/client/v4/zones/{zone}/purge_cache" \
  -H "Authorization: Bearer $CF_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{"files":["https://api.example.com/api/v1/products/42"]}'
```

Quand l'utiliser ? Pour toute API publique servant du contenu cacheable. Pas pour des réponses purement privées (sauf à utiliser `Cache-Control: private`).

2.1.3 Niveau 3 — Reverse proxy

Définition

Reverse proxy : serveur intermédiaire installé *devant* votre application, sur votre propre infrastructure. Reçoit les requêtes des clients, les transmet à l'origin, et peut conserver les réponses pour les requêtes suivantes identiques. Trois outils dominent : **nginx**, **Varnish**, **Traefik**.

Implémentation nginx :

Exemple

```
proxy_cache_path /var/cache/nginx levels=1:2
                 keys_zone=api_cache:50m # 50 MB de métadonnées
                 max_size=2g # 2 GB sur disque
                 inactive=60m;

server {
    listen 443 ssl http2;
    server_name api.example.com;

    location /api/v1/products {
        proxy_pass http://backend_pool;
        proxy_cache api_cache;
        proxy_cache_key "$scheme$request_method$host$request_uri";
        proxy_cache_valid 200 5m; # cacher 200 OK 5 minutes
        proxy_cache_valid 404 30s; # cacher 404 30 secondes
        proxy_cache_use_stale error timeout updating; # servir périmé en cas de panne
        add_header X-Cache-Status $upstream_cache_status;
    }

    location / { # non caché par défaut
        proxy_pass http://backend_pool;
    }
}
```

Vérification : après deux requêtes successives, observer l'en-tête `X-Cache-Status: MISS` puis `HIT`. La seconde réponse n'a jamais touché votre backend.

Varnish (plus puissant pour le cache, langage VCL dédié) :

Exemple

```
sub vcl_recv {
    if (req.url ~ "^/api/v1/products") {
        unset req.http.Cookie; # ignore cookies pour cacher
    }
}
sub vcl_backend_response {
    if (bereq.url ~ "^/api/v1/products") {
        set beresp.ttl = 5m;
        set beresp.grace = 1h; # servir périmé jusqu'à 1h si origin down
    }
}
```

2.1.4 Niveau 4 — Cache applicatif

Définition

Cache applicatif : stockage en mémoire vive (*RAM*) que votre application consulte avant la BDD. Permet de mémoriser des calculs coûteux, des résultats de requêtes lourdes, des sessions utilisateur. Trois outils principaux : **Redis** (le plus polyvalent), **Memcached** (cache KV pur), et un **cache local LRU** en mémoire du process.

Redis avec cache-aside :

Exemple

```
import Redis from "ioredis";
const redis = new Redis({ host: "redis.internal", port: 6379 });

async function getUserProfile(userId) {
  const key = `user:${userId}:profile`;

  // 1. Tenter le cache
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  // 2. Miss : aller en BDD
  const profile = await db.users.findById(userId);
  if (!profile) return null;

  // 3. Mémoriser pour la prochaine fois (TTL 5 min)
  await redis.setex(key, 300, JSON.stringify(profile));
  return profile;
}

async function updateUserProfile(userId, patch) {
  await db.users.update(userId, patch);
  // Invalider explicitement le cache pour ne pas servir l'ancien
  await redis.del(`user:${userId}:profile`);
}
```

Effet mesurable : un endpoint qui faisait 80 ms passe à 1–2 ms. La BDD voit son trafic chuter de 95 %.

Memcached (plus simple, sans persistance) :

Exemple

```
import Memcached from "memcached";
const mc = new Memcached("mc.internal:11211");
mc.set("user:42:profile", JSON.stringify(profile), 300, () => {});
mc.get("user:42:profile", (err, data) => { /* ... */ });
```

Quand préférer Memcached à Redis ? Si vous avez *uniquement* besoin d'un cache KV plat (pas de listes/sets/streams) et que la simplicité prime.

Cache LRU local (pour les données très chaudes, partagées dans le process) :

Exemple

```
import { LRUCache } from "lru-cache";
const featureFlags = new LRUCache({
  max: 1000, // max 1000 entrées
  ttl: 60 * 1000 // TTL 60 secondes
});

async function isFeatureEnabled(name, userId) {
  const key = `${name}:${userId}`;
  let value = featureFlags.get(key);
  if (value === undefined) {
    value = await db.flags.evaluate(name, userId);
    featureFlags.set(key, value);
  }
  return value;
```

```
}
```

Important : un cache local est par instance. Avec 10 pods K8s, vous avez 10 caches indépendants. Idéal pour des données fréquemment consultées qui changent peu (flags, configurations).

LRU = *Least Recently Used* : quand le cache est plein, on évince l'entrée la moins récemment consultée. Autres politiques courantes : LFU (*Least Frequently Used*), FIFO, ARC.

2.1.5 Niveau 5 — Cache base de données

Définition

Cache BDD : optimisations *internes* au SGBD (Système de Gestion de Base de Données) qui évitent de re-faire le même travail. L'application n'a rien à coder ; le moteur s'en occupe.

Trois mécanismes à distinguer :

Exemple

1. *Query cache* (résultats de requête). Postgres ne l'a pas natif (volontairement : invalidation impossible à garantir). MySQL l'avait jusqu'en 8.0 puis l'a retiré. À reproduire au niveau *applicatif* avec Redis si nécessaire.

2. *Plan cache* (plans d'exécution).

Quand vous envoyez une requête SQL à Postgres ou MySQL, le moteur ne l'exécute pas directement. Il passe par cinq phases :

1. **Parsing** : transformer le texte SQL en arbre syntaxique.
2. **Analyse** : résoudre les noms (tables, colonnes), vérifier les types.
3. **Réécriture** : appliquer vues, règles, sécurité au niveau ligne.
4. **Planification (*planning*)** : générer plusieurs *plans d'exécution* possibles (séquentiel scan, index scan, bitmap scan, jointures hash/merge/nested-loop…), estimer leur coût avec les statistiques, choisir le moins cher.
5. **Exécution** : appliquer le plan retenu et renvoyer les lignes.

Les phases 1–4 peuvent coûter **plus que l'exécution elle-même** pour une requête simple, surtout si la requête est complexe (5 jointures, sous-requêtes). Pour une requête répétée des milliers de fois par seconde avec juste des paramètres qui changent, c'est du gaspillage.

Le **plan cache** mémorise le plan retenu en phase 4 et le réutilise tel quel pour les appels suivants. La clé du cache est le texte SQL « squelette » (sans valeurs paramétrées).

Exemple

Sans préparation (chaque exécution paie le planning) :

```
SELECT * FROM orders WHERE user_id = 42 AND status = 'pending';
SELECT * FROM orders WHERE user_id = 99 AND status = 'shipped';
SELECT * FROM orders WHERE user_id = 123 AND status = 'pending';
-- 3 plannings -> 3 plans calculés
```

Avec préparation explicite (Postgres) :

```
PREPARE q (int, text) AS
  SELECT * FROM orders WHERE user_id = $1 AND status = $2;

EXECUTE q(42, 'pending'); -- planning fait ici (1ère fois)
EXECUTE q(99, 'shipped'); -- plan réutilisé
EXECUTE q(123, 'pending'); -- plan réutilisé
DEALLOCATE q; -- libérer le plan
```

Via le driver pg de Node :

```
// Mode "named" : le driver demande implicitement PREPARE
await client.query({
  name: "find-orders", // ce nom déclenche le plan cache
  text: "SELECT * FROM orders WHERE user_id = $1 AND status = $2",
  values: [42, "pending"]
});
```

Effet mesurable : pour une requête simple à 5 jointures, le planning coûte typiquement 1–3 ms ; l'exécution 0,5 ms. Avec plan cache, on supprime les 1–3 ms et la latence chute d'un facteur 3 à 6.

Piège des plans paramétrés. Postgres bascule en *generic plan* après 5 exécutions, qui peut être moins optimal que les plans *custom* (avec valeurs constantes). Si vos paramètres ont des distributions très différentes (`status='pending'` a 10 lignes, `status='shipped'` a 10 millions), un seul plan ne conviendra pas aux deux. Solutions : `plan_cache_mode = force_custom_plan`, ou ne pas préparer ces requêtes-là.

3. *Materialized view* (vue matérialisée). Une vue normale est recalculée à chaque appel ; une vue *matérialisée* stocke physiquement le résultat et peut être rafraîchie périodiquement.

```
-- Vue matérialisée : calcul lourd (jointures + agrégat)
CREATE MATERIALIZED VIEW daily_sales AS
SELECT product_id, DATE(created_at) AS day, SUM(amount) AS total
FROM orders
WHERE status = 'shipped'
GROUP BY product_id, DATE(created_at);

CREATE UNIQUE INDEX ON daily_sales (product_id, day);

-- Rafraîchissement (1 fois par heure via cron)
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales;
```

Effet : une requête analytique qui prenait 30 s en agrégeant à la volée prend désormais 50 ms en lisant la vue matérialisée.

2.2 Stratégies de cache : quatre patterns

Le « niveau » dit *où* se trouve le cache ; la *stratégie* dit *comment* il interagit avec la base de vérité (BDD).

2.2.1 Cache-aside (lazy loading)

Définition

Cache-aside : l'application est responsable du cache. Elle consulte le cache d'abord ; au *miss* elle charge depuis la BDD et écrit dans le cache. C'est le pattern le plus utilisé (90 % des cas) car il est simple et donne le contrôle total.

Exemple

```
async function read(key) {
  let v = await cache.get(key); // 1. cache
  if (v) return v; // HIT -> retour direct
  v = await db.find(key); // 2. miss -> BDD
  if (v) await cache.set(key, v, 300); // 3. on remplit le cache
  return v;
}
async function write(key, patch) {
  await db.update(key, patch);
  await cache.del(key); // invalidation après écriture
}
```

Avantages : simple, robuste à une panne du cache (juste plus lent). **Inconvénient :** données potentiellement périmées entre une écriture concurrente et l'invalidation.

2.2.2 Read-through

Définition

Read-through : le *cache lui-même* est responsable du chargement depuis la BDD au miss. L'application ne parle qu'au cache, qui sait quoi faire si la clé manque. Nécessite un cache « intelligent » (DAX, Hazelcast, certains Redis modules).

Exemple

Avec AWS DynamoDB Accelerator (DAX) : votre code ne change pas, il appelle DynamoDB normalement.

```
const dax = new AmazonDaxClient({ endpoints: ["dax.internal:8111"] });
const item = await dax.getItem({ TableName: "users", Key: { id: 42 } });
// DAX répond direct si en cache, sinon va lire DynamoDB en interne et garnit le cache.
```

Avantage : code applicatif inchangé, transparent. **Inconvénient :** lié à un produit commercial, moins de contrôle fin.

2.2.3 Write-through

Définition

Write-through : toute écriture passe par le cache, qui écrit ensuite synchroniquement en BDD. Garantit que cache et BDD sont *toujours* cohérents au prix d'une écriture plus lente.

Exemple

```
async function write(key, value) {
  await cache.set(key, value);
  await db.upsert(key, value); // bloquant -> attente que la BDD réponde
  // si la BDD échoue, on doit défaire le cache (gestion d'erreur)
}
```

Quand l'utiliser ? Quand on veut éviter à tout prix de servir une valeur périmée *après* une écriture (panier d'achat, statut de commande).

2.2.4 Write-behind (write-back)

Définition

Write-behind : l'écriture va dans le cache, qui répond immédiatement, puis pousse vers la BDD de manière *asynchrone* (par lots). Très rapide en perception utilisateur mais risque de perte de données si le cache crashe avant flush.

Exemple

Cas typique : compteurs de vues d'un article (incrémentation très fréquente, faible criticité).

```
// chaque vue : juste +1 dans Redis
await redis.incr(`views:article:${id}`);

// toutes les 60 secondes, un job flush vers la BDD
setInterval(async () => {
  const keys = await redis.keys("views:article:*");
  for (const k of keys) {
    const id = k.split(":")[2];
    const delta = await redis.getdel(k);
    if (delta) await db.articles.incrementViews(id, Number(delta));
  }
}, 60_000);
```

Risque : si Redis crashe, on perd les 60 dernières secondes de vues. Acceptable pour un compteur ; inacceptable pour des écritures bancaires.

2.2.5 Variante — Refresh-ahead

Définition

Refresh-ahead : le cache rafraîchit *proactivement* les clés très consultées avant qu'elles n'expirent. Évite que la première requête après expiration paie le coût du rechargement.

Implémentation simple : un *cron* (job planifié) qui, toutes les 4 minutes, recharge les TOP 100 clés (TTL = 5 min).

2.3 Cache HTTP : Cache-Control, ETag, Last-Modified

Trois en-têtes HTTP régissent le cache. Tous trois doivent être maîtrisés ensemble.

2.3.1 En-tête `Cache-Control`

Définition

Cache-Control (RFC 9111) : en-tête de réponse HTTP qui décrit la politique de cache souhaitée. Une liste de *directives* séparées par des virgules indique aux intermédiaires (navigateur, CDN, reverse proxy) ce qu'ils ont le droit de faire avec la réponse.

Directives essentielles à connaître :

- `public` : tout intermédiaire peut cacher (CDN inclus).
- `private` : seul le client final peut cacher (interdit aux CDN). À utiliser pour les réponses spécifiques à un utilisateur.
- `max-age=N` : durée en secondes pendant laquelle la réponse est considérée fraîche par les clients.
- `s-maxage=N` : équivalent pour les caches *partagés* (CDN, proxy). Si présent, prime sur `max-age` chez eux.
- `no-cache` : à ne pas confondre avec `no-store`. Signifie « cache OK mais revalide à chaque fois ».
- `no-store` : interdit absolument tout stockage. À utiliser pour les données très sensibles (login, paiement).
- `must-revalidate` : une fois expiré, ne pas servir la version périmée même si l'origin est down.
- `stale-while-revalidate=N` : autorise à servir la version périmée pendant N secondes *en attendant* qu'une revalidation termine. Excellente UX.
- `immutable` : la ressource ne changera *jamais* (pour les bundles JS versionnés type `app-3a4f.js`). Le navigateur n'envoie même pas de revalidation.

Exemple

Réponse complète avec directives combinées :

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=60, s-maxage=300,
               stale-while-revalidate=86400
Vary: Accept, Accept-Encoding
```

Lecture humaine : « cette réponse peut être cachée par tout le monde ; le client la considère fraîche 60 s ; le CDN la considère fraîche 300 s ; et tout le monde peut continuer à servir la version périmée jusqu'à 24 h, le temps de re-fetcher derrière. »

2.3.2 En-tête `ETag` (Entity Tag)

Définition

ETag (*Entity Tag*, RFC 9110) : identifiant opaque qui représente une *version particulière* d'une ressource. Le serveur l'attache à chaque réponse ; à la prochaine requête, le client le renvoie via `If-None-Match`, et le serveur peut répondre `304 Not Modified` (vide) si la version est inchangée. Économie de bande passante : on évite de re-télécharger ce qui n'a pas bougé.

Comment générer un ETag ? Le plus souvent, un hash du contenu (SHA-1 tronqué, MD5) ou un compteur de version stocké en BDD. Doit être stable tant que la ressource ne change pas.

Fort vs faible. Un ETag *fort* (`"abc123"`) signifie « byte-à-byte identique ». Un ETag *faible* préfixé `W/` (`W/"abc123"`) signifie « sémantiquement équivalent » (par exemple, on autorise l'ordre des champs JSON à varier). Préférer `W/` pour le JSON.

Exemple

Round-trip économe :

```
# Premier appel
GET /articles/42 HTTP/1.1
-> HTTP/1.1 200 OK
   ETag: W/"sha1-3a4f...e8"
   Content-Length: 2340
   { ... contenu JSON ... }

# Deuxième appel (10 minutes plus tard)
GET /articles/42 HTTP/1.1
If-None-Match: W/"sha1-3a4f...e8"
-> HTTP/1.1 304 Not Modified <-- aucun corps renvoyé
   ETag: W/"sha1-3a4f...e8"
```

Gain : 2,3 KB économisés par requête. Sur un client mobile en 4G dégradée, le 304 est aussi 5 à 10 fois plus rapide.

2.3.3 En-tête `Last-Modified`

Définition

Last-Modified : variante plus ancienne et plus simple d'ETag, fondée sur une *date* de dernière modification. Le serveur renvoie `Last-Modified: <date>` ; le client renvoie `If-Modified-Since: <date>` ; si la ressource n'a pas changé depuis, le serveur répond `304`.

Limites par rapport à ETag :

- Précision d'une seconde seulement (deux modifications dans la même seconde sont indiscernables).
- Suppose une horloge fiable côté serveur.
- Marche mal si plusieurs serveurs servent la même ressource avec des dates de modification différentes.

En pratique en 2026 : utiliser ETag par défaut ; Last-Modified comme complément ou pour des ressources statiques (fichiers).

Exemple

```
GET /articles/42 HTTP/1.1
-> HTTP/1.1 200 OK
   Last-Modified: Tue, 05 May 2026 10:14:02 GMT
   ETag: W/"sha1-3a4f...e8"

# 1 heure plus tard
GET /articles/42 HTTP/1.1
If-Modified-Since: Tue, 05 May 2026 10:14:02 GMT
If-None-Match: W/"sha1-3a4f...e8"
-> HTTP/1.1 304 Not Modified
```

Bonne pratique : envoyer les deux. Le client revalide avec ce qu'il a.

2.4 Cache stampede — piège à connaître

Définition

Cache stampede : quand une clé expire sous forte charge, N requêtes simultanées tombent en cache miss et appellent toutes la BDD en parallèle. Conséquence : la BDD s'effondre brièvement.

Solutions :

- *Mutual exclusion* : verrou Redis (`SET NX EX`), seul le premier reconstruit.
- *Refresh anticipé probabiliste* : recalcul avant l'expiration formelle.
- *Background refresh* : un processus séparé rafraîchit les clés chaudes.

3 Compression et connexions

3.1 Compression HTTP

Définition

Compression HTTP : technique où le serveur compresse le corps de la réponse avant l'envoi et le client le décompresse à réception. Négociée par les en-têtes `Accept-Encoding` (client) et `Content-Encoding` (serveur). Gain typique sur du JSON ou du HTML : 70–85 % de taille en moins.

3.1.1 gzip

Définition

gzip (*GNU Zip*, 1992) : algorithme de compression sans perte, dérivé de DEFLATE (LZ77 + Huffman). Universel : tous les navigateurs et clients HTTP le supportent. Excellent compromis vitesse/taux.

3.1.2 Brotli (`br`)

Définition

Brotli (Google, 2015) : algorithme plus récent qui compresse 10 à 25 % mieux que gzip pour le texte. Supporté par tous les navigateurs modernes (Chrome, Firefox, Safari, Edge depuis 2017). Identifiant HTTP : `br`.

Exemple

Comparaison concrète sur un payload JSON de 100 KB (catalogue produits) :

Algorithme	Taille compressée	Réduction
Aucun (`identity`)	100 000 octets	—
gzip — niveau 6 (défaut)	22 340 octets	77,7 %
gzip — niveau 9 (max)	21 870 octets	78,1 %
brotli — niveau 4	18 210 octets	81,8 %
brotli — niveau 11 (max)	16 430 octets	83,6 %

Configuration nginx pour activer Brotli avec fallback gzip :

```
# /etc/nginx/nginx.conf (module ngx_brotli requis)
brotli on;
brotli_comp_level 4; # bon compromis CPU/taille
brotli_types application/json text/plain text/html
                  text/css application/javascript;
brotli_min_length 1024; # ne pas compresser < 1 KB

gzip on;
gzip_comp_level 6;
gzip_types application/json text/plain text/html
                  text/css application/javascript;
gzip_min_length 1024;
```

Comportement : si le client envoie `Accept-Encoding: br, gzip`, nginx choisit Brotli ; s'il envoie `Accept-Encoding: gzip`, nginx tombe en gzip ; sinon réponse non compressée.
Vérification :

```
curl -H "Accept-Encoding: br, gzip" \
     -I https://api.example.com/products
# Réponse contient : Content-Encoding: br
```

3.1.3 Règles pratiques

- Seuil : ne pas compresser sous **1 KB** (l'overhead de compression et les en-têtes coûtent plus que ce qu'on gagne).
- Ne pas compresser ce qui est déjà compressé (JPEG, PNG, MP4, ZIP, fichiers déjà gzip). Double compression = perte de temps et parfois fichier plus gros.
- Compresser tous les formats texte : JSON, HTML, CSS, JavaScript, XML, SVG, CSV.
- Attention à l'attaque **BREACH** (*Browser Reconnaissance and Exfiltration via Adaptive Compression of Hypertext*, 2013) : ne pas inclure dans une même réponse compressée un secret *et* une donnée contrôlée par l'attaquant. Séparer les pages publiques et privées.

3.2 Versions de HTTP et performance

- HTTP/1.1 : connexions persistantes, mais limite à 6 connexions par origine.
- HTTP/2 : multiplexage natif (plusieurs flux par connexion), HPACK (compression des en-têtes).
- HTTP/3 : repose sur QUIC (UDP), supprime le head-of-line blocking TCP. Latence améliorée sur réseaux mobiles.

4 Scalabilité horizontale

4.1 Stateless = scalable

Définition

Stateless : un service est sans état si chaque requête contient toute l'information nécessaire à son traitement, sans dépendre d'une mémoire interne. Conséquence : on peut répliquer le service N fois sans coordination.

Tout l'état doit être externalisé :

- Sessions : Redis ou cookie signé.
- Cache : Redis (partagé entre instances).
- Fichiers temporaires : S3 ou équivalent.
- Locks : Redis ou Postgres advisory locks.

4.2 Load balancing

Définition

Load balancer (LB) : composant placé devant un groupe d'instances (*backends* ou *upstreams*) qui répartit le trafic entrant entre elles. Trois rôles inséparables :

- **Scaler horizontalement** : on peut ajouter N backends, le LB répartit.
- **Tolérer les pannes** : si un backend tombe, le LB l'exclut du pool.
- **Lisser la charge** : aucun backend n'est sur-sollicité tant qu'un autre est libre.

4.2.1 L4 vs L7 — deux couches de routage

Définition

Couche 4 (*L4*, transport, TCP/UDP) : le LB décide en regardant uniquement IP source/destination et numéros de port. Très rapide (millions de paquets/seconde), aveugle au contenu applicatif. Exemples : HAProxy en mode TCP, AWS NLB (*Network Load Balancer*), nginx `stream`.

Couche 7 (*L7*, application, HTTP/HTTPS) : le LB termine le TLS, lit la requête HTTP et décide sur méthode/URL/headers. Plus lent (limité par CPU et terminaison TLS), mais beaucoup plus de fonctionnalités. Exemples : nginx, HAProxy en mode HTTP, AWS ALB (*Application Load Balancer*), Envoy, Traefik.

Exemple

Cas L4 typique : équilibrer du trafic gRPC ou WebSocket pur sans inspection. Exemple AWS NLB.

```
# nginx en mode L4 (stream)
stream {
  upstream postgres_pool {
    server db1.internal:5432;
    server db2.internal:5432;
    server db3.internal:5432;
  }
  server {
    listen 5432;
    proxy_pass postgres_pool;
  }
}
```

Cas L7 typique : routing par path vers différents services.

```
# nginx en mode L7 (HTTP)
http {
  upstream orders_api { server o1:3000; server o2:3000; }
  upstream users_api { server u1:3000; server u2:3000; }
  upstream static_app { server s1:3000; }

  server {
```

```
    listen 443 ssl http2;
    server_name api.example.com;

    location /api/v1/orders { proxy_pass http://orders_api; }
    location /api/v1/users { proxy_pass http://users_api; }
    location / { proxy_pass http://static_app; }
  }
}
```

4.2.2 Algorithmes de répartition

Exemple

Round-robin (« chacun son tour ») : on attribue les requêtes 1, 2, 3 aux backends A, B, C, puis 4 à A, 5 à B, etc. Le plus simple. Bon si les backends sont identiques et les requêtes de coût similaire.

```
upstream pool {
  server a.internal:3000;
  server b.internal:3000;
  server c.internal:3000;
  # round-robin par défaut
}
```

Least connections : envoie la prochaine requête au backend qui a actuellement le moins de connexions ouvertes. Adapté quand les durées de traitement varient.

```
upstream pool {
  least_conn;
  server a.internal:3000;
  server b.internal:3000;
}
```

IP hash : la même IP source tape toujours le même backend (*sticky session*). Utile si chaque backend a un cache local. Mauvais pour l'équité (un NAT d'entreprise envoie tout le bureau sur un seul backend).

```
upstream pool {
  ip_hash;
  server a.internal:3000;
  server b.internal:3000;
}
```

Weighted : on pondère selon la capacité. Utile pour une migration progressive ou pour exploiter des machines hétérogènes.

```
upstream pool {
  server gros.internal:3000 weight=3; # 3 fois plus de trafic
  server petit.internal:3000 weight=1;
}
```

Power of two choices (P2C) : tire deux backends au hasard, envoie au moins chargé des deux. Très efficace, peu coûteux en mesure. C'est l'algorithme par défaut d'Envoy et NGINX Plus.
Adaptive / latency-aware : utilise la latence p99 observée par backend pour choisir le plus rapide. Présent dans HAProxy 3, Envoy.

4.2.3 Health checks — le LB doit savoir qui est vivant

Définition

Health check : sonde périodique envoyée par le LB à chaque backend pour vérifier qu'il est en état de servir. Si N échecs consécutifs, le backend est marqué *down* et retiré du pool ; il revient automatiquement après M succès.

Exemple

Convention : l'application expose un endpoint `/healthz` qui répond 200 si tout va bien.

```
app.get("/healthz", async (req, res) => {
  // Vérifier les dépendances critiques
  try {
    await db.query("SELECT 1"); // BDD répond
    await redis.ping(); // cache répond
    res.json({ status: "ok" });
  } catch (e) {
    res.status(503).json({ status: "degraded" });
  }
});
```

Configuration nginx active health check (NGINX Plus ou `ngx_http_upstream_module` avec patch) :

```
upstream pool {
  server a.internal:3000;
  server b.internal:3000;
  server c.internal:3000;
}
server {
  location / {
    proxy_pass http://pool;
    health_check uri=/healthz interval=5s fails=3 passes=2;
  }
}
```

Effet : si le backend `b` renvoie 3 fois 503 d'affilée, nginx l'exclut du pool. Toutes les requêtes vont sur `a` et `c`. Au bout de 2 succès, `b` revient.

4.2.4 Exemple complet : 3 backends + load balancer + bench

Exemple

Architecture :

```
                  +-- backend-1 (3000)
client -> [nginx] -+-- backend-2 (3000)
                  +-- backend-3 (3000)
```

docker-compose.yml :

```
services:
  api1: { image: my-api:1.0, ports: ["3001:3000"] }
  api2: { image: my-api:1.0, ports: ["3002:3000"] }
  api3: { image: my-api:1.0, ports: ["3003:3000"] }
  lb:
    image: nginx:alpine
    ports: ["80:80"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on: [api1, api2, api3]
```

nginx.conf (mode L7, least connections, keep-alive vers backends) :

```
upstream api_pool {
  least_conn;
```

```
  keepalive 32;
  server api1:3000 max_fails=3 fail_timeout=10s;
  server api2:3000 max_fails=3 fail_timeout=10s;
  server api3:3000 max_fails=3 fail_timeout=10s;
}

server {
  listen 80;
  location / {
    proxy_pass http://api_pool;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
```

Pour observer la répartition, chaque backend renvoie son propre identifiant :

```
// Dans my-api (Node)
const POD = process.env.HOSTNAME; // nom du container
app.get("/", (req, res) => res.json({ pod: POD }));
```

```
for i in $(seq 1 6); do curl -s http://localhost/ ; done
# {"pod":"api1"}
# {"pod":"api2"}
# {"pod":"api3"}
# {"pod":"api1"} # round-robin sur connexions équitables
# {"pod":"api2"}
# {"pod":"api3"}
```

4.3 Health checks

Définition

Liveness : « le process est-il vivant ? » (sinon, redémarrer).
Readiness : « le process est-il prêt à servir ? » (sinon, retirer du LB).

4.4 Autoscaling

- **HPA** (*Horizontal Pod Autoscaler*) Kubernetes : scale sur CPU, mémoire ou métriques custom.
- **KEDA** : autoscaling sur événements externes (queue Kafka, files SQS, requêtes Prometheus).

5 API Gateways

Définition

API Gateway : composant placé en amont d'un ensemble d'API qui centralise les fonctions transverses : routing, authentification, rate limiting, transformation, observabilité.

Pourquoi en utiliser une ?

- Centraliser ce qui ne devrait pas être dans chaque service métier.

- Faciliter la migration vers une nouvelle version (routing).
- Appliquer une politique de sécurité homogène.
- Activer l'observabilité une seule fois pour toutes les API.

Solutions. Kong, Tyk, KrakenD, Apache APISIX (open source). AWS API Gateway, Azure API Management, GCP API Gateway, Apigee (cloud). Cloudflare, Fastly (edge).

6 Throttling et rate limiting

6.1 Cinq notions à distinguer

Définition

Rate limiting : plafonner le nombre d'opérations sur une fenêtre de temps (ex. 100 req/sec).

Throttling : politique *douce* qui ralentit (queue, retry-after) plutôt que de refuser durement.

Quota : limite cumulée sur une période longue (ex. 10 000 req/mois). Souvent associé à la facturation.

Spike arrest : protection contre les pics instantanés.

Concurrency limit : nombre maximum de requêtes *en cours simultanément.*

6.2 Sur quel axe ?

- Par IP, par utilisateur authentifié, par clé API, par tenant, par endpoint, par méthode HTTP, par tier de plan, ou global.

6.3 Algorithmes de rate limiting

- **Token Bucket** : un seau de jetons qui se remplit à débit constant. Tolère les bursts. *Le plus utilisé.*
- **Leaky Bucket** : sortie à débit constant. Lisse strictement le trafic.
- **Fixed Window Counter** : compteur réinitialisé à intervalle fixe. Simple mais souffre du burst à la frontière.
- **Sliding Window Counter** : combine deux fenêtres consécutives. Précis et économe en mémoire. *Standard de fait.*

6.4 Headers HTTP standardisés

Exemple

Réponse 429 (Too Many Requests, RFC 6585) :

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1728478800

{
  "type": "https://api.example.com/errors/rate-limit",
  "title": "Trop de requêtes",
  "status": 429,
```

```
  "detail": "Limite de 1000 req/min dépassée. Réessayez dans 60s."
}
```

6.5 Côté client : être un bon citoyen

Recevoir un 429 n'est pas une fatalité : un bon client sait s'adapter et ne pas aggraver la situation. Quatre techniques essentielles.

6.5.1 1. Respecter `Retry-After`

Définition

`Retry-After` : en-tête HTTP envoyé par le serveur en réponse à un 429 ou un 503, indiquant le nombre de secondes (ou la date HTTP) à attendre avant de retenter. *C'est un ordre, pas une suggestion.*

Exemple

```
async function callWithRetryAfter(url, opts) {
  const res = await fetch(url, opts);
  if (res.status === 429 || res.status === 503) {
    const retryAfter = parseInt(res.headers.get("retry-after") || "1", 10);
    console.log(`Throttled, retry in ${retryAfter}s`);
    await sleep(retryAfter * 1000);
    return callWithRetryAfter(url, opts); // rejouer
  }
  return res;
}
```

Ne pas faire : ignorer `Retry-After` et retenter immédiatement. Le serveur vous bannira plus durement.

6.5.2 2. Exponential backoff avec jitter

Définition

Exponential backoff : technique où le délai d'attente *double* (ou triple) à chaque tentative ratée. Première erreur → attendre 100 ms ; deuxième → 200 ms ; troisième → 400 ms ; quatrième → 800 ms… Mathématiquement : $\text{delay}_n = \text{base} \cdot 2^{n-1}$.

Jitter (« gigue ») : ajout d'un *aléa* au délai pour casser la synchronisation entre clients. Sans jitter, après une panne d'API, les 10 000 clients qui ont tous échoué à la même seconde retentent tous à la seconde suivante : c'est le *thundering herd* (« troupeau qui charge »), qui maintient le serveur à terre.

Trois variantes de jitter (article AWS « Exponential Backoff and Jitter », 2015) :

Exemple

Backoff exponentiel sans jitter (mauvais) :

```
function delay(attempt) {
  return Math.min(100 * Math.pow(2, attempt), 30000);
```

```
}
// attempt=0 -> 100ms, 1 -> 200ms, 2 -> 400ms, 3 -> 800ms ...
```

Problème : tous les clients ratent au même instant et retentent au même instant.
Full jitter (recommandé) :

```
function delay(attempt) {
  const cap = Math.min(100 * Math.pow(2, attempt), 30000);
  return Math.random() * cap; // entre 0 et cap
}
// attempt=2 (cap=400) -> entre 0 et 400ms aléatoire
```

Effet : les clients sont étalés uniformément sur la fenêtre. Plus de troupeau.
Equal jitter (compromis : moitié fixe, moitié aléatoire) :

```
function delay(attempt) {
  const cap = Math.min(100 * Math.pow(2, attempt), 30000);
  return cap / 2 + Math.random() * (cap / 2);
}
```

Decorrelated jitter (Google, encore plus efficace en panne longue) :

```
let last = 100;
function delay() {
  last = Math.min(30000, Math.random() * last * 3 + 100);
  return last;
}
```

6.5.3 3. Fonction de retry complète

Exemple

```
async function callWithRetry(fn, { maxAttempts = 5, baseMs = 100 } = {}) {
  let lastErr;
  for (let i = 0; i < maxAttempts; i++) {
    try {
      const res = await fn();
      // Erreurs serveur ou throttling -> retry
      if (res.status === 429 || res.status >= 500) {
        // Honorer Retry-After s'il est présent
        const retryAfter = res.headers.get("retry-after");
        if (retryAfter) {
          await sleep(parseInt(retryAfter, 10) * 1000);
          continue;
        }
        throw new Error(`HTTP ${res.status}`);
      }
      // Erreurs client (4xx hors 429) -> ne pas retenter
      if (res.status >= 400) return res;
      return res; // succès
    } catch (err) {
      lastErr = err;
      if (i === maxAttempts - 1) break;
      const cap = Math.min(baseMs * 2 ** i, 30_000);
      const wait = Math.random() * cap; // full jitter
      console.log(`Attempt ${i + 1} failed, retry in ${wait.toFixed(0)}ms`);
      await sleep(wait);
```

```
    }
  }
  throw lastErr;
}

// Usage : seulement sur opérations idempotentes
const products = await callWithRetry(
  () => fetch("https://api.example.com/products")
);
```

Règle absolue : ne retenter automatiquement que des opérations *idempotentes* (GET, PUT, DELETE) ou des POST avec un *Idempotency-Key*. Sinon risque de double paiement, double commande, double mail.

6.5.4 4. Lire `X-RateLimit-Remaining` pour anticiper

Exemple

```
const res = await fetch(url);
const remaining = parseInt(res.headers.get("x-ratelimit-remaining") || "1000", 10);
const resetAt = parseInt(res.headers.get("x-ratelimit-reset") || "0", 10);

if (remaining < 50) {
  // On approche du quota -> ralentir préventivement
  const secondsUntilReset = resetAt - Math.floor(Date.now() / 1000);
  const targetDelay = (secondsUntilReset * 1000) / remaining;
  await sleep(targetDelay);
}
```

Effet : on lisse soi-même le débit pour éviter de se faire `429`-er.

6.5.5 5. Préférer les API en lot

Au lieu de 1 000 appels `POST /orders`, un appel `POST /orders/bulk` avec 1 000 objets dans le body. La plupart des API publiques (Stripe, Shopify, GitHub) facturent au nombre d'appels, pas au nombre d'objets traités : on réduit la charge sur le serveur et son propre quota d'un coup.

7 Patterns de résilience

7.1 Timeout

Toujours plafonner. Cascade : le timeout du caller doit être inférieur au timeout du callee, plus une marge.

7.2 Retry + backoff + jitter

- N tentatives (typiquement 3).
- Délai exponentiel : 200 ms, 400 ms, 800 ms…
- Jitter pour éviter le *thundering herd* (rafale synchronisée).
- Uniquement sur opérations idempotentes (ou avec *Idempotency-Key*).

7.3 Circuit breaker

Définition

Circuit breaker : pattern qui « ouvre un circuit » (refuse en bloc) après N erreurs consécutives, pour laisser le service amont récupérer. Trois états : *Closed* (passant), *Open* (refus), *Half-Open* (test).

Bibliothèques : Resilience4j (Java), Polly (.NET), opossum (Node), tenacity (Python).

7.4 Bulkhead

Définition

Bulkhead (« cloison étanche ») : isoler les ressources par type d'appel pour qu'une saturation ne se propage pas. Pools de threads ou pools de connexions séparés.

7.5 Fallback

Réponse dégradée plutôt qu'erreur : ancienne valeur de cache, valeur par défaut, message d'attente.

7.6 Idempotency-Key

Header propriétaire (Stripe, Square) qui permet à un client de rejouer une requête sans risque de doublon. Le serveur stocke la clé et associe la réponse précédemment calculée.

8 Les outils du module

8.1 Redis

Outil

Redis (`https://redis.io`) : base en mémoire ultra-rapide, structures de données riches (strings, hashes, lists, sets, sorted sets, streams). Standard de fait pour cache applicatif, sessions, locks distribués, files temporaires, et rate limiting.

8.2 Memcached

Outil

Memcached (`https://memcached.org`) : alternative historique à Redis, encore très utilisée. Plus simple (cache KV pur), pas de structures complexes ni de persistance. Très rapide.

8.3 nginx et Varnish

Outil

nginx : sert aussi de cache HTTP (directive `proxy_cache`). Très polyvalent.

Varnish (`https://varnish-cache.org`) : reverse proxy spécialisé dans le cache HTTP, langage de configuration *VCL*. Standard pour les sites à très fort trafic.

8.4 Kong, Tyk, KrakenD, Apache APISIX

Outil

API Gateways open source. Tous gèrent routing, auth, rate limiting, transformation, observabilité.
Kong : Lua + nginx, écosystème de plugins très large.
Tyk : Go, dashboard convivial.
KrakenD : Go, design *stateless*.
Apache APISIX : Lua + nginx, performances très élevées, projet ASF.

8.5 k6

Outil

k6 (`https://k6.io`, Grafana Labs) : outil moderne de *load testing*. Scénarios scriptables en JavaScript. Rapports CLI lisibles, intégration Grafana Cloud.

Exemple :

```
import http from "k6/http";
import { sleep } from "k6";
export const options = { vus: 50, duration: "60s" };
export default function () {
  http.get("https://api.example.com/products");
  sleep(0.1);
}
```

8.6 Outils CLI de bench HTTP

Définition

Bench HTTP en ligne de commande : ces outils envoient un grand nombre de requêtes vers une URL pendant une durée donnée, mesurent latences et débit, et impriment un rapport. Indispensables pour valider une optimisation, comparer deux versions, ou estimer un dimensionnement avant de mettre en charge avec k6.

8.6.1 wrk — débit maximal en ligne de commande

Outil

wrk (`https://github.com/wg/wrk`, par Will Glozer, 2012) : écrit en C avec event loop `epoll`/`kqueue`. Capable d'injecter des centaines de milliers de requêtes par seconde depuis une seule machine. Scriptable en Lua pour personnaliser les requêtes.

Installation : `apt install wrk` (Ubuntu/Debian), `brew install wrk` (macOS).

Exemple basique — 12 threads, 400 connexions, 30 secondes :

```
wrk -t12 -c400 -d30s --latency https://api.example.com/products

# Sortie typique :
# Running 30s test @ https://api.example.com/products
# 12 threads and 400 connections
# Thread Stats Avg Stdev Max +/- Stdev
```

```
# Latency 62.3ms 14.2ms 321ms 71.2%
# Req/Sec 0.55k 0.10k 1.2k 66.8%
# Latency Distribution
# 50% 58.1ms
# 75% 71.0ms
# 90% 82.4ms
# 99% 125.7ms
# 197 432 requests in 30.01s, 89.40MB read
# Requests/sec: 6 580.20
# Transfer/sec: 2.98MB
```

Bench POST avec script Lua (`post.lua`) :

```
-- post.lua
wrk.method = "POST"
wrk.body = '{"name":"test"}'
wrk.headers["Content-Type"] = "application/json"
wrk.headers["Authorization"] = "Bearer xyz"
```

```
wrk -t4 -c100 -d10s -s post.lua https://api.example.com/items
```

8.6.2 hey — l'équivalent moderne d'Apache Bench

Outil

hey (https://github.com/rakyll/hey, par Jaana Dogan, ex-Google) : écrit en Go, alternative simple à `ab` (*Apache Bench*). Sortie lisible immédiatement.

Installation : `go install github.com/rakyll/hey@latest`, ou télécharger le binaire.

Exemple — 10 000 requêtes, concurrence 50 :

```
hey -n 10000 -c 50 https://api.example.com/products

# Sortie :
# Summary:
# Total: 2.4521 secs
# Slowest: 0.1820 secs
# Fastest: 0.0021 secs
# Average: 0.0122 secs
# Requests/sec: 4078.61
# Latency distribution:
# 10% in 0.0048 secs
# 50% in 0.0110 secs
# 90% in 0.0218 secs
# 99% in 0.0512 secs
# Status code distribution:
# [200] 9998 responses
# [500] 2 responses
```

Avec headers personnalisés et payload :

```
hey -n 5000 -c 20 -m POST \
    -H "Authorization: Bearer xyz" \
    -H "Content-Type: application/json" \
    -d '{"name":"test"}' \
    https://api.example.com/items
```

Quand préférer hey ? Quand on cherche un rapport rapide à lire sans rentrer dans Lua ou JS. Idéal pour un test `Make`, `npm script`, ou en CI.

8.6.3 Vegeta — scénarios constants et rapports HTML

Outil

Vegeta (https://github.com/tsenart/vegeta, par Tomás Senart) : outil Go conçu autour du concept de *requests-per-second constant*. À la différence de wrk (qui pousse autant que possible), Vegeta cible un débit fixe (*rate*) et mesure la latence à ce débit — exactement ce qu'on veut pour modéliser une charge attendue.

Installation : `brew install vegeta` ou `go install github.com/tsenart/vegeta/v12@latest`.

Exemple — 200 req/sec pendant 1 minute :

```
echo "GET https://api.example.com/products" \
  | vegeta attack -rate=200 -duration=60s \
  | tee results.bin \
  | vegeta report

# Sortie :
# Requests [total, rate, throughput] 12000, 200.02, 199.84
# Duration [total, attack, wait] 60.045s, 60s, 45.1ms
# Latencies [min, mean, 50, 90, 95, 99, max]
# 11.2ms, 38.7ms, 32.1ms, 64.3ms, 81.0ms, 142.5ms, 521ms
# Bytes In [total, mean] 24 000 000, 2000
# Status Codes [code:count] 200:11985 500:15
```

Rapport HTML interactif (graphe latence dans le temps) :

```
vegeta report -type=hdrplot results.bin > latency.html
```

Cibler plusieurs URL (fichier de cibles) :

```
# targets.txt
GET https://api.example.com/products
POST https://api.example.com/items
Content-Type: application/json
@payload.json
```

```
vegeta attack -targets=targets.txt -rate=100 -duration=30s | vegeta report
```

Quand préférer Vegeta ? Quand on veut prouver « tient-elle 200 req/sec pendant 1 minute ? » plutôt que « quel est son débit max ? ». La quasi-totalité des SLO se mesure ainsi.

8.6.4 Comparatif rapide

Outil	Langue	Force	Faiblesse
wrk	C	Débit max très élevé, Lua scriptable	Pas de mode rate constant
hey	Go	Sortie lisible, install facile	Moins riche en fonctionnalités
Vegeta	Go	Mode rate constant, rapport HTML	Configuration plus verbeuse
ab	C	Préinstallé partout	Obsolète, biais sur HTTPS
k6	JS	Vrais scénarios, intégration Grafana	Plus lourd, plus complexe

Règle pratique : `hey` pour 5 minutes de validation, `wrk` pour tester un débit max, `Vegeta` pour mesurer la latence à débit constant, `k6` pour des scénarios complexes (login + parcours utilisateur).

8.7 Cloudflare, Fastly, CloudFront, Akamai

Outil

CDN (*Content Delivery Network*) : réseau de serveurs distribués géographiquement. Servent du contenu cacheable au plus près de l'utilisateur. Tous gèrent aussi WAF, anti-DDoS, terminaison TLS, et pour certains des fonctions edge (Cloudflare Workers, Fastly Compute@Edge).

8.8 Resilience4j, Polly, opossum, tenacity

Outil

Bibliothèques de patterns de résilience.
Resilience4j : Java moderne, modulaire (CircuitBreaker, RateLimiter, Retry, Bulkhead, Timeout).
Polly : .NET, syntaxe fluente.
opossum : Node, circuit breaker.
tenacity : Python, retry avec stratégies.

9 Synthèse

À retenir

- Toujours raisonner en **percentiles** ($p_{95}, p_{99}, p_{99.9}$), jamais en moyenne.
- Cache à plusieurs étages, toujours définir la stratégie d'invalidation.
- **Stateless = scalable** : externaliser session, cache, fichiers.
- Compression et HTTP/2/3 par défaut. Attention à BREACH si secrets dans la réponse.
- Throttling = combinaison rate limit + quota + spike arrest + concurrency limit.
- Résilience par défaut : timeout, retry+backoff+jitter, circuit breaker, bulkhead, fallback.

10 Lexique du module

API Gateway Point d'entrée centralisé pour plusieurs API.

Backpressure Mécanisme pour signaler au producteur de ralentir.

BREACH Attaque exploitant la compression HTTPS pour extraire des secrets.

Bulkhead Isolation de ressources pour confiner les pannes.

Cache-aside Stratégie où l'application gère le cache (lazy loading).

CDN Content Delivery Network.

Circuit breaker Pattern qui ouvre un circuit après N erreurs.

Compression Réduction de la taille du contenu (gzip, brotli).

Concurrency limit Plafond du nombre de requêtes simultanées en cours.

ETag Identifiant de version d'une ressource (cache HTTP).

Fallback Réponse dégradée en cas d'échec.

GCRA Generic Cell Rate Algorithm. Variante de leaky bucket.

HPA Horizontal Pod Autoscaler (Kubernetes).

Idempotency-Key En-tête pour rejouer une requête sans doublon.

Jitter Délai aléatoire ajouté à un backoff.

k6 Outil de load testing moderne.

Latence Temps de traitement d'une requête.

L4/L7 Couches transport (TCP) et application (HTTP) du modèle OSI.

Leaky bucket Algorithme de rate limit à débit de sortie constant.

Load balancer Répartiteur de trafic.

Loi de Little $L = \lambda \cdot W$: concurrence = débit × latence.

Memcached Cache KV en mémoire historique.

Percentile p_N : valeur en-deçà de laquelle se trouvent $N\,\%$ des observations.

Quota Limite cumulée sur période longue.

Rate limiting Plafond instantané de débit.

ReDoS Regular Expression Denial of Service.

Redis Base en mémoire polyvalente.

Resilience4j Bibliothèque Java de résilience.

Retry-After Header HTTP indiquant la durée à attendre avant de retenter.

Sliding window Algorithme de rate limit à fenêtre glissante.

Spike arrest Protection contre les pics instantanés.

Stateless Sans état serveur entre requêtes.

Sticky session Routage qui force un client vers le même backend.

Throttling Régulation du débit (souvent douce).

Token bucket Algorithme de rate limit avec tolérance aux bursts.

Varnish Reverse proxy spécialisé dans le cache HTTP.

11 Pour aller plus loin

- Michael Nygard, *Release It!*, 2nd ed.
- Ilya Grigorik, *High Performance Browser Networking* (gratuit en ligne).
- Martin Kleppmann, *Designing Data-Intensive Applications*.
- RFC 6585 (HTTP 429), RFC 9110 (HTTP Semantics).
- Documentation Redis, k6, Cloudflare.